AI시스템 반도체 SW 개발_2기_Cotex-M4프로젝트 보고서

(6팀) 남성현, 조성수, 김형진

깃허브 저장소 https://github.com/SCCI-STM/PRJ1

목차

- 구현 목표 및 개요
  - 1. 프로젝트 개요
  - 2. 개발 환경 및 사양
- 요구사항 정의
  - 1. 기능 요구사항
  - 2. 비기능 요구사항
  - 3. 제약사항
- 시스템 아키텍처
  - 1. 시스템 동작 흐름도 (Flowchart)
  - 2. 상태 전이도 (State Machine Diagram)
- 하드웨어 설계
  - 1. 사용 부품
  - 2. Pin Assignment
- 소프트웨어 설계
  - 1. 모듈 구조 및 파일 목록
  - 2. 이벤트 목록
  - 3. 발생 인터럽트 목록
  - 4. 변수 설정
  - 5. 함수 설정
  - 6. main.c - 메인함수 상세
  - 7. exception.c - 인터럽트 제어 상세
  - 8. motor.c - 레지스터 기반 모터 제어 상세
- 파형 출력 결과
- 개인 트러블 슈팅 및 후기(남성현)
- 개인 트러블 슈팅 및 후기(조성수)
- 개인 트러블 슈팅 및 후기(김형진)

구현 목표 및 개요

1. 프로젝트 개요

본 프로젝트는 STM32F4(ARM Cortex-M4) 기반 보드와 L293 모터 드라이버를 이용하여 DC 모터의 정방향 회전, 역방향 회전, 정지 및 PWM 속도 제어 기능을 구현하는 것을 목표로 한다.

2. 개발 환경 및 사양

구분	항목	내용 및 사양	비고
HW	MCU	STM32F4(ARM Cortex-M4)	
HW	Motor Driver	L293	
HW	Actuator	DC Motor	
HW	UART/Power Module	CP2102 Module	모터 전원 공급
SW	Language	C	
SW	IDE	Visual Studio Code	

요구사항 정의

1. 기능 요구사항

1.1 버튼 제어

- 시스템 초기화가 완료될 때까지 모터는 정지 상태를 유지해야 한다.
- 버튼을 3초 미만으로 눌렀다가 놓으면 모터의 회전 방향을 변경해야 한다.
- 버튼을 3초 이상 누르면 모터를 정지해야 한다.

1.2 UART 제어

- `F` 또는 `f`를 수신하면 모터를 정방향(`CW`)으로 구동해야 한다.
- `R` 또는 `r`을 수신하면 모터를 역방향(`CCW`)으로 구동해야 한다.
- `S` 또는 `s`를 수신하면 모터를 정지해야 한다.
- `1`부터 `9`까지의 숫자를 수신하면 입력 단계에 따라 모터의 PWM Duty를 변경해야 한다.
- 정의되지 않은 UART 명령은 무시해야 한다.

1.3 방향 전환 보호

- 모터가 회전 중인 상태에서 반대 방향으로 전환될 경우 즉시 역회전하지 않아야 한다.
- 모터를 먼저 정지시킨 후 1초가 지나면 반대 방향으로 구동해야 한다.

2. 비기능 요구사항

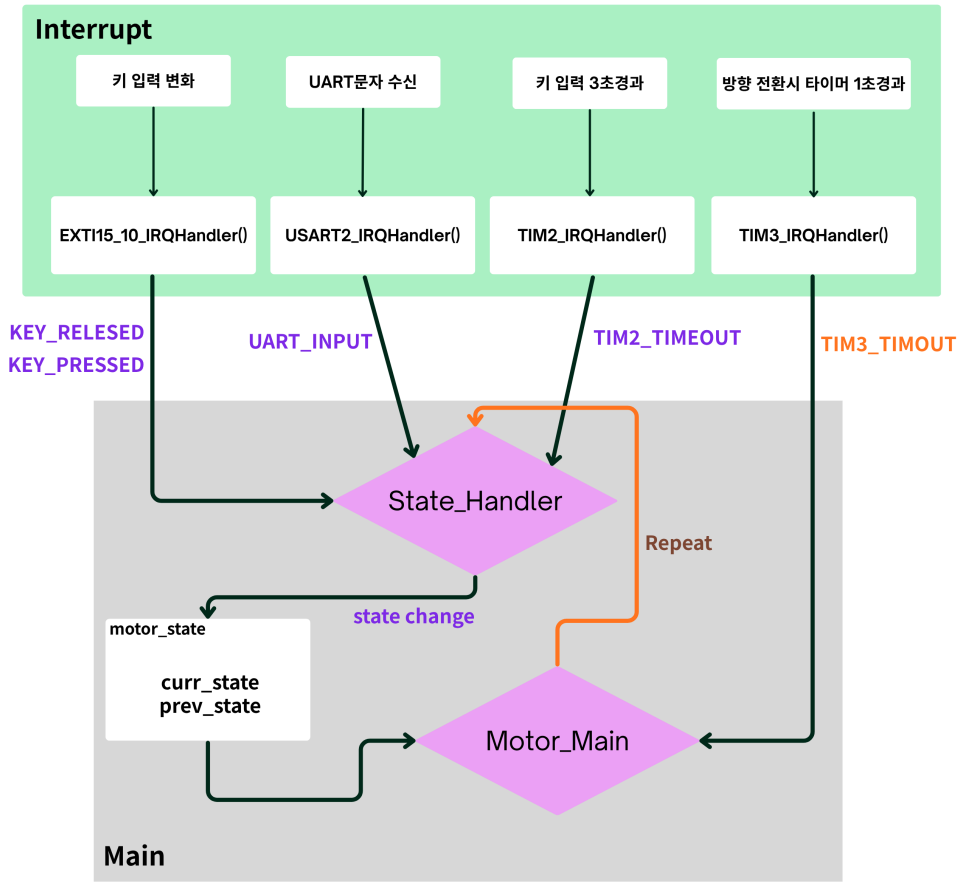
- 버튼 및 UART 입력 처리와 방향 전환을 위한 1초 정지 구간에도 메인 루프는 중단되지 않고 지속적으로 동작해야 한다.
- 모터의 급격한 방향 전환으로 인한 과전류 발생과 모터 드라이버의 손상을 방지해야 한다.

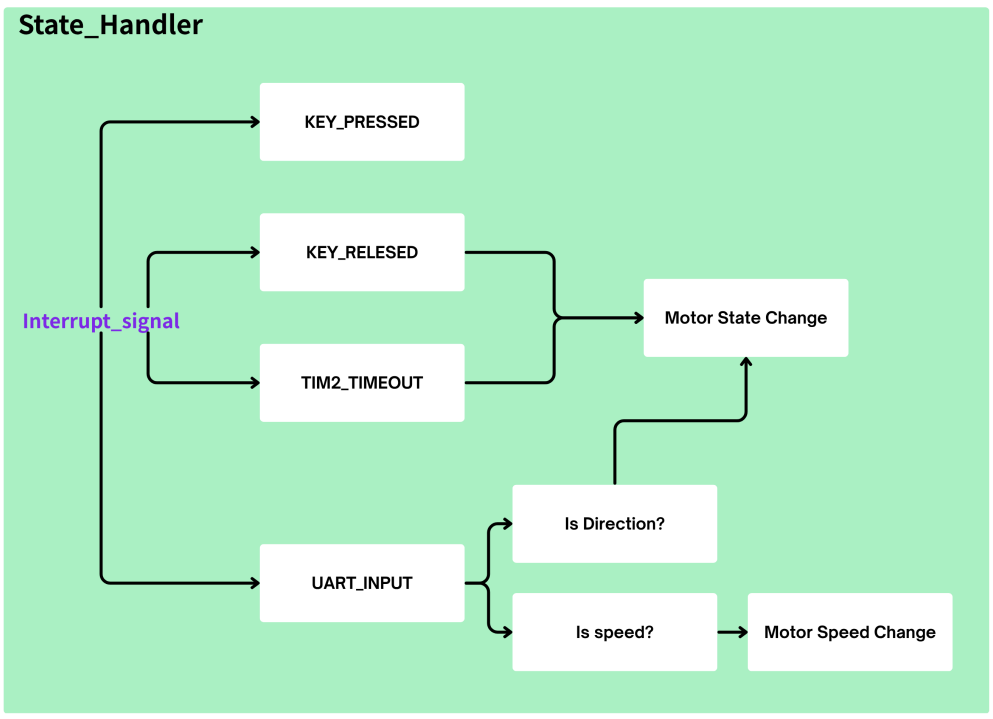
3. 제약사항

- L293의 모터 전원은 CP2102 모듈의 `+3.3V` 또는 `+5V`를 이용하여 연결해야 한다.
- PWM 주파수와 PWM Duty Rate는 실험을 통해 적절한 값으로 설정해야 한다.
- PWM Duty Rate가 너무 낮으면 모터가 구동되지 않을 수 있으므로, Duty 범위는 `50%~100%`로 제한해야 한다.

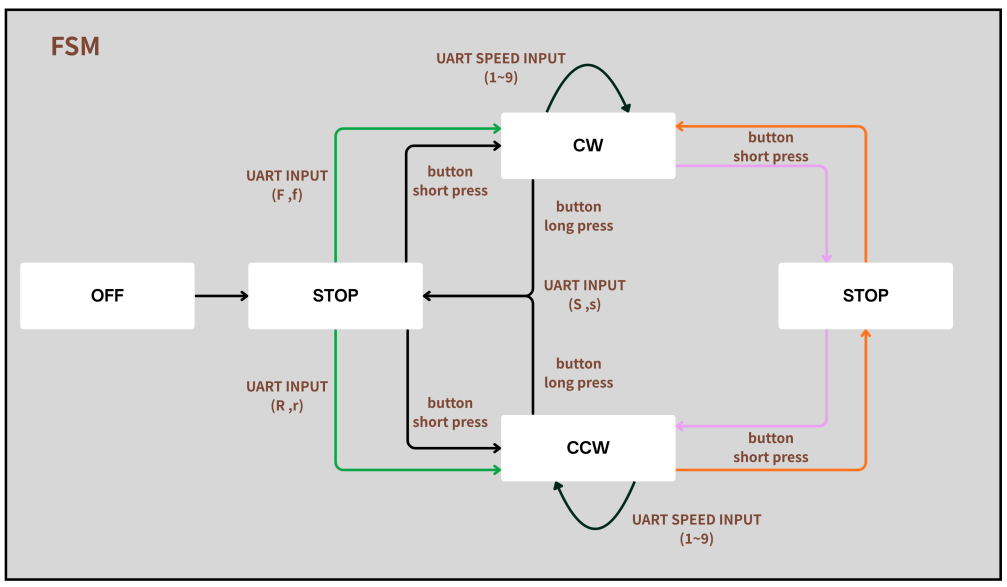
시스템 아키텍처

1. 시스템 동작 흐름도 (Flowchart)





2. 상태 전이도 (State Machine Diagram)



하드웨어 설계

1. 사용 부품

- **MCU**: STM32 M4 Board
- **Motor Driver**: L293 Driver IC
- **Actuator**: DC Motor

2. Pin Assignment

Peripheral	Pin / Channel	역할 (Function)
GPIO Pin	PA0	Motor Driver Line 1 (1A) / TIM5 CH1
GPIO Pin	PA1	Motor Driver Line 2 (2A) / TIM5 CH2
EXTI	PC13	Push Button (Falling / Rising Edge 감지)
USART2	PA2 (TX), PA3 (RX)	Baudrate 115200 외부 시리얼 명령어 수신
TIM2	Internal	3초 Long Press 감지용 One-shot Timer
TIM3	Internal	방향 전환 시 1초 비동기 딜레이 체크용 Repeat Timer
TIM5	Internal	PA0, PA1 PWM Signal Generator (AF02)

소프트웨어 설계

1. 모듈 구조 및 파일 목록

파일	역할 (Function)
main.c	메인 루프 동작
exception.c	인터럽트 예외처리
timer.c	타이머2,3,5 설정
uart.c	uart통신 설정
motor.c	모터제어 설정
key.c	버튼 설정
device_driver.h	공통헤더파일

2. 이벤트 목록

이벤트 구분	이벤트	발생 조건	처리 내용	상태 변화
None	없음			
Key	`Key_Pressed`	키를 눌렀을 때	`short`, `long` 입력인지 판단을 위한 3초 측정을 시작한다.	
Key	`Key_Relesed`	키를 땠을 때	`short` 입력으로 판단하면 다음 회전 방향을 결정하고 long입력으로 판단하면 넘어간다	`short`= `CW` ↔ `CCW`, `long` = 넘어감
Timer	`TIMER2_OUT`	3초가 지났을 때	3초가 지나도록 키를 누르고 있으면 `long` 입력으로 판단	
UART	`UART_INPUT`	UART로 입력이 들어 왔을 때	입력 데이터에 따라 모터상태 변경 or 속도변경	

3. 발생 인터럽트 목록

- **EXTI15_10_ISR**: PC13 비트를 확인하여 Key Push 시 `TIM2` 스타트, Key Release 시 `TIM2` 스탑 및 Short Press 판단
- **TIM2_ISR**: 3초 이상 누르고 있을 경우 `is_long_pressed = 1` 및 `TIMER2_OUT` 이벤트 발생
- **TIM3_ISR**: 1초 지연 인터럽트 발생 시 `TIM3_Expired = 1` 플래그 세팅
- **USART2_ISR**: 수신 데이터 유형 분류 (`1`: 방향 명령어, `2`: 속도 단계)

4. 변수 설정

파일명	구분	특수키워드	타입	변수(상수)명	초기값	역할
device.driver.h	정의			PWM_INPUT_FREQ	10000	PWM 파형 주파
device.driver.h	정의			MOTOR_SPEED_INIT	50	모터 초기
device.driver.h	정의			MOTOR_SPEED_UP	5	모터 속도 증가 단위
device.driver.h	메크로			MOTOR_SPEED_STEP(x)	MOTOR_SPEED_INIT + (MOTOR_SPEED_UP * x)	모터속도 자동 계산
timer.c	정의			TIM2_TICK	20	Timer2 한 클록의 시간(usec)
timer.c	정의			TIM2_FREQ	1000000/TIM2_TICK	Timer2 주파수 50000Hz
timer.c	정의			TIM2_PLS_OF_1ms	1000/TIM2_TICK	Timer2의 1ms당 클록 수 (50)

파일명	구분	특수키워드	타입	변수(상수)명	초기값	역할
timer.c	정의			TIM2_MAX	0xffffu	Timer2에서 ARR에 저장되는 최대값
timer.c	정의			TIM2_INIT	3000	3초
timer.c	정의			TIM3_TICK	20	Timer3 한 클록의 시간(usec)
timer.c	정의			TIM3_FREQ	1000000/TIM3_TICK	Timer3 주파수 50000Hz
timer.c	정의			TIM3_PLS_OF_1ms	1000/TIM3_TICK	Timer3의 1ms당 클록 수 (50)
timer.c	정의			TIM3_1sec	1000	1초
timer.c	정의			TIM5_TICK	20	Timer5 한 클록의 시간(usec)
timer.c	정의			TIM5_FREQ	1000000/TIM5_TICK	Timer5 주파수 50000Hz
timer.c	정의			TIM5_PLS_OF_1ms	1000/TIM5_TICK	Timer5의 1ms당 클록 수 (50)

파일명	구분	구조체 타입	상수명
device_driver.h	상태 구조체	eventState	NONE
device_driver.h	상태 구조체	eventState	KEY_PRESSED
device_driver.h	상태 구조체	eventState	KEY_RELEASED
device_driver.h	상태 구조체	eventState	UART_INPUT
device_driver.h	상태 구조체	eventState	TIMER2_OUT
device_driver.h	상태 구조체	MotorState	MOTOR_STOP
device_driver.h	상태 구조체	MotorState	MOTOR_CW
device_driver.h	상태 구조체	MotorState	MOTOR_CCW

파일명	구분	특수 키워드	타입	변수명	초기값	역할
main.c	상태변수		eventState	event	NONE	어떤 이벤트가 발생했는지
main.c	상태변수	volatile	int	is_long_pressed	0	키가 3초이상 눌렸는지
main.c	상태변수	volatile	int	is_key_pressed	0	키를 누르고 있는 상태인지
main.c	상태변수	volatile	int	uart_data_in	0	uart로 데이터가 들어왔다면 무슨 타입인지
main.c	데이터변수	volatile	unsigned char	input_motor_dir	0	모터가 어떤 방향인지
main.c	데이터변수	volatile	int	input_motor_speed	0	Uart로 요청한 모터 속력

파일명	구분	특수 키워드	타입	변수명	초기값	역할
motor.c	상태변수	static	MotorState	running_state	MOTOR_STOP	현재 동작하는 상태를 저장
motor.c	플래그 변수	volatile	int	TIM3_Expired	0	타이머3번이 타임아웃됬는지 확인하는 변수

5. 함수 설정

파일명	타입	함수이름	매개변수	반환값	역할
uart.c	함수	Uart2_Init	int baud	void	UART2 통신 속도(Baud rate) 초기화
uart.c	함수	Uart2_RX_Interrupt_Enable	int en	void	UART2 수신 인터럽트 활성화/비활성화

파일명	타입	함수이름	매개변수	반환값	역할
key.c	함수	Key_Init	void	void	키/버튼 입력을 위한 GPIO 초기화
key.c	함수	Key_ISR_Enable	int en	void	키 인터럽트(EXTI) 활성화/비활성화

파일명	타입	함수이름	매개변수	반환값	역할
timer.c	함수	Timer2_Init	void	void	타이머 2 초기화

파일명	타입	함수이름	매개변수	반환값	역할
timer.c	함수	Timer2_Start	void	void	타이머 2 동작 시작
timer.c	함수	Timer2_Stop	void	void	타이머 2 동작 정지
timer.c	함수	TIM2_Interrupt_Enable	int en	void	타이머 2 인터럽트 활성화/비활성화
timer.c	함수	TIM3_Repeat_Interrupt_Enable	int en	void	타이머 3 반복 인터럽트 활성화/비활성화
timer.c	함수	Timer5_Init	void	void	타이머 5 초기화
timer.c	함수	Timer5_Out_Pwm_Generator	int duty	void	지정한 듀티비(Duty Cycle)로 타이머 5 PWM 출력 제어

파일명	타입	함수이름	매개변수	반환값	역할
motor.c	함수	Motor_Init	void	void	모터 제어 신호 및 핀 초기화
motor.c	함수	Motor_Main	void	void	모터 동작 메인 루틴
motor.c	함수	Long_key_motor_state	void	void	키 입력 지속 시간에 따른 모터 상태 전환
motor.c	함수	Motor_ProcessKeyState	void	void	키 입력 상태에 따른 모터 동작 제어
motor.c	함수	Motor_ProcessUartState	char data	void	UART 수신 데이터에 따른 모터 동작 제어
motor.c	함수	Motor_Stop	void	void	모터 동작 정지
motor.c	함수	Motor_RotateCW	void	void	모터 시계 방향(CW) 회전
motor.c	함수	Motor_RotateCCW	void	void	모터 반시계 방향(CCW) 회전

파일명	타입	함수이름	매개변수	반환값	역할
exception.c	ISR	EXTI15_10_IRQHandler	void	void	외부 인터럽트(버튼) 서비스 루틴
exception.c	ISR	TIM2_IRQHandler	void	void	타이머 2 인터럽트 서비스 루틴
exception.c	ISR	TIM3_IRQHandler	void	void	타이머 3 인터럽트 서비스 루틴
exception.c	ISR	USART2_IRQHandler	void	void	USART2 통신 인터럽트 서비스 루틴

6. main.c - 메인함수 상세

```
for(;;)
{
    ------------------------------------------------------------------
    state_handler()
        - if(이벤트 == None)  return

        - else if(이벤트 == Key Pressed)
            event = None

        - else if(이벤트 == timer2_out)
            길게 눌렀을때 함수
            event = None

        - else if(이벤트 == Key released)
            if (!is_long_pressed)
              Motor_ProcessKeyState();
              Timer5_Out_Pwm_Generator(MOTOR_SPEED_STEP(0));

            event = None

        - if (이벤트 == uart 입력 플래그 변수)
                if 플래그 1
                    Motor_ProcessUartState(INPUT_MOTOR_DIR)
                else if 플래그 2
                    Timer5_Out_Pwm_Generator(MOTOR_SPEED_STEP(INPUT_MOTOR_SPEED))
                event = None
    ------------------------------------------------------------------

    motor_main();

}
```

7. exception.c - 인터럽트 제어 상세

```
---------------------------------------------------------------------
타이머 3번 인터럽트
3번 타이머 를 써서  1초 체크되면 1
---------------------------------------------------------------------
키인터럽트

    키눌렀을때 인터럽트
    - 이벤트 = KEY_PRESSED
    - is_key_pressed = 1
    - is_long_pressed = 0
    - 타이머2 3초 스타트

    키뗐을때 인터럽트
    - 이벤트상태 = KEY_RELEASED
    - is_key_pressed = 0
    - 타이머2 스톱
---------------------------------------------------------------------
3초가 지났을때 인터럽트
- if (is_key_pressed) {
            이벤트 = timer2_out
            is_long_pressed=1
  }
---------------------------------------------------------------------
UART가 들어왔을때 인터럽트
if 일단 입력받은거를   ('S''s' / 'F''f' / 'R''r') 이거면
- INPUT_MOTOR_DIR = DR
- 플래그 1

else if 입력이 숫자 1~9라면
- INPUT_MOTOR_SPEED = DR
- 플래그 2

else
아니라면 플래그 0으로
```

8. motor.c - 레지스터 기반 모터 제어 상세

```
PA0, PA1 alternate_mode 사용시
타이머 5번의 1,2번 채널을 사용한다

===정지상태일때=== Motor_Stop
GPIOA->MODER => 0번핀 1번 핀 모두 01 01
GPIO->ODR => 0;

====1 0일때===== //Motor_RotateCW
GPIOA->MODER => 10 01 //한쪽만 주변장치모드로 해서 타이머 5번의 주파수를 출력해야한다
GPIOA->AFR[0] => 4번비트에 2(타이머 5번)를 줘야한다
GPIO->ODR =>  0번비트에 0
TIM5->CCMR1 -> pwm모드(14~12번비트) 110고정

====0 1일때===== //Motor_RotateCCW
GPIOA->MODER => 01 10
GPIOA->AFR[0] => 0번자리에 2(타이머 5번)를 줘야한다
GPIO->ODR =>  1번비트에 0
TIM5->CCMR1 -> pwm모드(4~6번비트) 110고정

---------------------------------------------------------------------
Long_key_motor_state()
prev = curr
curr = stop

---------------------------------------------------------------------
Motor_ProcessKeyState()
curr -> 현재 돌고 있는 상태
prev -> 이전에 돌고있던 상태

우리는 curr을 보고 버튼을 눌렀을때 curr 반대방향으로 방향이 바뀐다는점

1. curr == stop
    1-1. if 처음 상태라면  prev == stop
              curr = cw, prev = cw
```

```
    1-2. else if prev == cw
                curr_state = cw;
        prev_state = stop;

  1-3. else if prev == ccw
                curr_state = ccw;
        prev_state = stop

2. curr == cw
    2-1 curr == cw, prev == ccw -> curr = ccw, prev = cw 저장

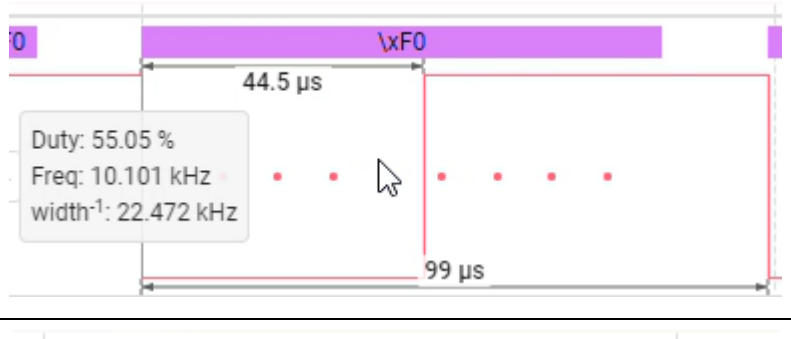
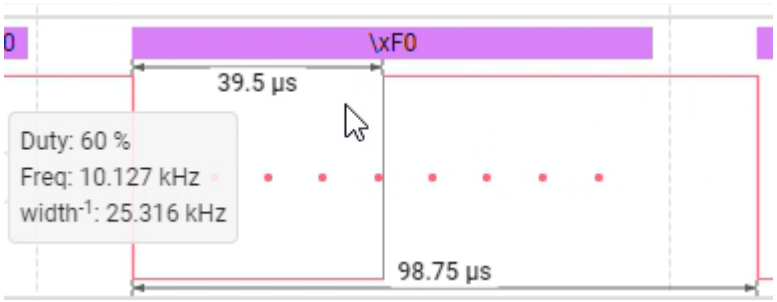
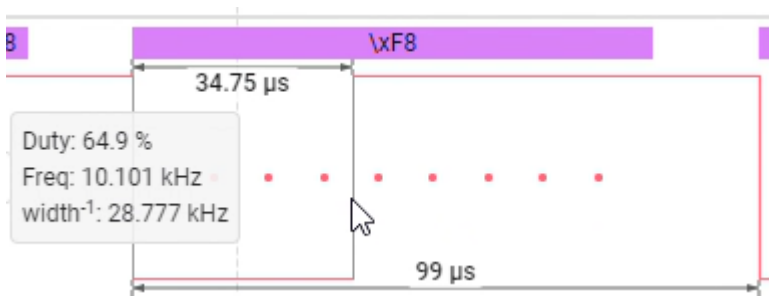
3. curr == ccw
    3-1 curr == ccw, prev == cw -> curr = cw, prev = ccw 저장

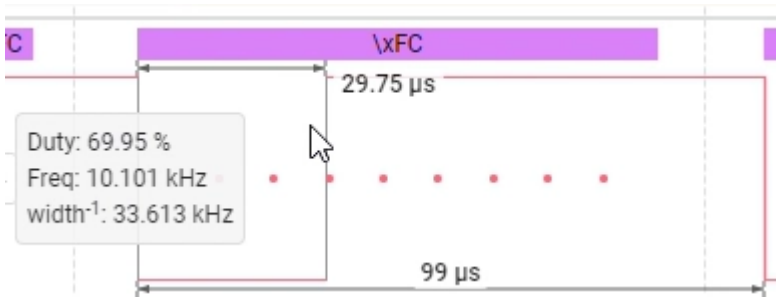
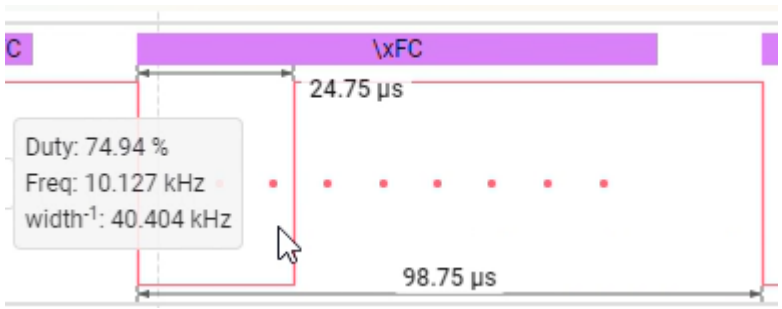
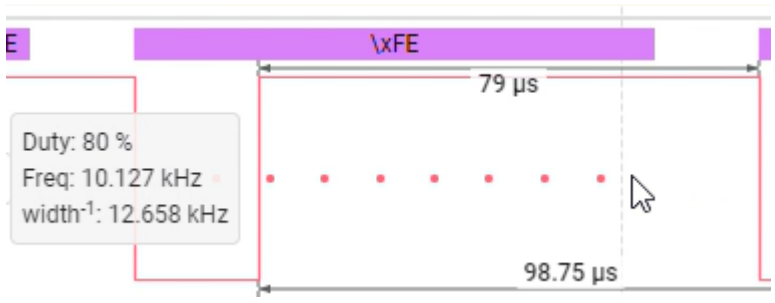
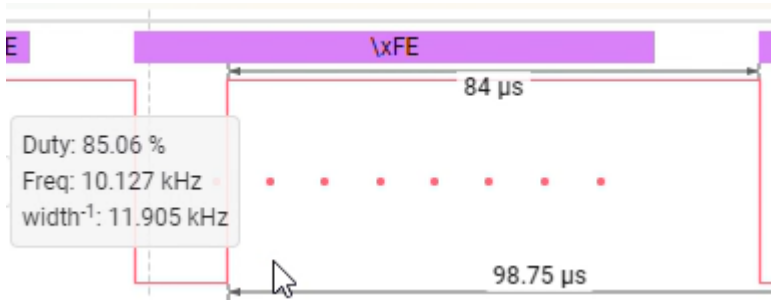
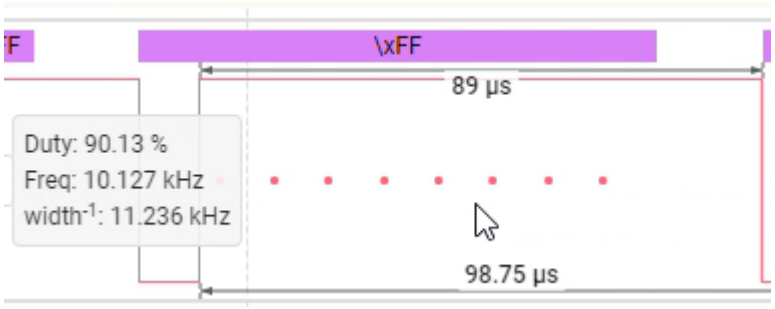
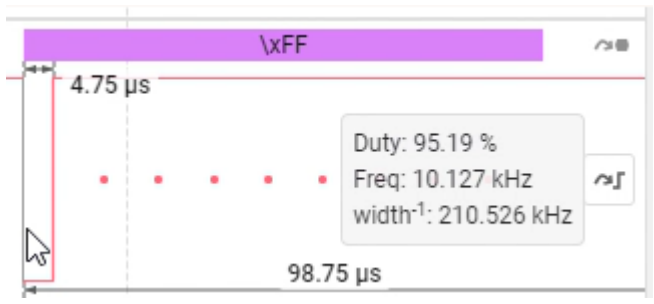

-----------------------------------------------------------------------
Motor_ProcessUartState(INPUT_DATA)
prev = curr
if F
- curr = cw

else if s
- curr = stop

else if r
- curr = ccw
```

파형 출력 사진

속도	속도파형 이미지	듀티
0		50%
1		55%
2		60%
3		65%

속도	속도파형 이미지	듀티
4		70%
5		75%
6		80%
7		85%
8		90%
9		95%

개인 트러블 슈팅 및 후기(남성현)

상위 설계서와 다른 구현 사항

1. 모터 정지 출력

- 상위 설계: PA0과 PA1을 모두 Low로 출력
- 현재 구현: PA0과 PA1을 모두 High로 출력

두 방식 모두 L293 입력을 같은 레벨로 만들어 모터를 정지시킨다. 현재 구현은 실제 회로에서 확인한 `11` 정지 방식을 사용한다.

2. CW/CCW의 MODER 값

상태	상위 설계	현재 구현
`MOTOR_CW`	`0x9`	`0x6`
`MOTOR_CCW`	`0x6`	`0x9`

차이는 실제 배선 방향을 반영한 결과이다.

3. UART 속도 입력 범위

- 상위 설계: 문자 `1~9`
- 현재 구현: 문자 `0~9`

현재 속도 정책은 50%부터 95%까지 총 10단계이므로 `0~9` 입력을 사용한다.

4. USART2 수신 데이터 분류 위치

- 상위 설계: `USART2_IRQHandler()`에서 방향 명령과 속도 명령을 분류
- 현재 구현: ISR에서는 수신 데이터 저장과 이벤트 발생만 수행하고, `main.c`의 `state_handler()`에서 명령을 분류

5. TIM3 동작 모드

- 상위 설계: Repeat Timer
- 현재 구현: One-pulse mode

```
TIM3->CR1 = (1 << 4) | (1 << 3);
```

6. 1초 안전 지연의 적용 범위

상위 설계의 핵심 요구사항은 CW와 CCW 사이의 방향 전환 전에 1초간 정지하는 것이다. 현재 구현은 `curr_state != applied_state`이면 항상 안전 지연을 적용하므로 다음 상황에도 1초 지연이 발생한다.

- 부팅 후 최초 구동
- 정지 후 재구동
- CW와 CCW 사이의 방향 전환

즉, 현재 구현은 방향 전환뿐 아니라 모든 정지 상태에서 구동 상태로 진입할 때 안전 지연을 적용한다.

7. Short Press 후 PWM Duty

- 상위 설계: Short Press 시 방향을 토글하고 Duty를 초기값으로 설정
- 현재 구현: 방향만 토글하고 기존 TIM5 CCR 값을 유지

현재 `EVENT_KEY_RELEASED` 처리에서는 다음 함수만 호출한다.

```
Motor_ProcessKeyState();
```

따라서 UART로 설정한 속도는 키로 방향을 바꿔도 유지된다. Short Press 때 항상 50%로 초기화하려면 아래 호출을 추가해야 한다.

```
Timer5_Out_Pwm_Generator(MOTOR_SPEED_STEP(0));
```

8. TIM5 초기 Duty 및 시작 시점

`Timer5_Init()`은 PSC, ARR, CCMR1, CCER를 설정하지만 `CCR1`, `CCR2`에 초기 Duty를 넣거나 `CEN`을 활성화하지 않는다. TIM5의 실제 PWM 시작은 `Timer5_Out_Pwm_Generator()`가 처음 호출될 때 이루어진다.

```
Timer5_Init()
    → TIM5 기본 설정만 수행

UART 숫자 입력
    → CCR1/CCR2 설정
    → EGR.UG 발생
    → TIM5.CEN 활성화
```

따라서 방향 명령이나 키 입력 전에 속도 명령을 한 번도 받지 않았다면, 초기 PWM 상태가 입력 순서에 따라 달라질 수 있다. 입력 순서와 관계없이 항상 같은 초기 속도로 시작하려면 시스템 초기화 후 다음 함수를 호출하는 방식을 사용할 수 있다.

```
Timer5_Out_Pwm_Generator(MOTOR_SPEED_INIT);
```

트러블슈팅

1. 버튼을 누르자마자 Long Press로 처리되는 문제

증상

버튼을 누르면 `KEY_PRESSED`가 먼저 처리되지 않고 TIM2 인터럽트가 즉시 발생해 Timeout 상태로 처리되었다.

원인

TIM2의 PSC와 ARR 값을 반영하기 위해 `EGR.UG`로 강제 Update Event를 발생시켰다. 이 과정에서 `SR.UIF`가 설정될 수 있으며, Update Interrupt가 활성화된 상태라면 곧바로 TIM2 인터럽트 요청이 발생할 수 있다.

해결

타이머 시작 전에 다음 순서로 기존 상태를 정리했다.

```
Macro_Set_Bit(TIM2->EGR, 0);        // UG 발생
Macro_Clear_Bit(TIM2->SR, 0);       // UIF Clear
NVIC_ClearPendingIRQ(TIM2_IRQn);    // NVIC Pending Clear
Macro_Set_Bit(TIM2->DIER, 0);       // Update Interrupt Enable
Macro_Set_Bit(TIM2->CR1, 0);        // Timer Start
```

`Timer2_Stop()`에서는 `DIER.UIE`도 비활성화하여 정지 상태에서 불필요한 Update Interrupt 요청이 발생하지 않도록 했다.

2. 속도 단계가 정상적으로 증가하지 않는 문제

증상

- 속도 0단계에서 모터가 동작하지 않음
- 0단계에서 1단계로 변경할 때 속도가 급격히 증가함
- 1단계 이후에는 단계별 속도 차이를 체감하기 어려움

원인

속도 단계 계산식의 초기 속도와 증가 폭 위치가 서로 뒤바뀌어 있었다.

수정 전:

```
#define MOTOR_SPEED_STEP(x) \
    ((MOTOR_SPEED_UP) + ((MOTOR_SPEED_INIT) * (x)))
```

이 식은 `x=0`일 때 5%, `x=1`일 때 55%, `x=2`일 때 105%가 되어 정상적인 Duty 단계가 만들어지지 않는다.

해결

다음과 같이 초기 속도에 단계별 증가량을 더하도록 수정했다.

```
#define MOTOR_SPEED_STEP(x) \
    ((MOTOR_SPEED_INIT) + ((MOTOR_SPEED_UP) * (x)))
```

수정 후 입력 `0`~`9`가 Duty 50%~95%에 대응한다.

회고

- 설계 단계에서 함수와 변수의 세부 구현까지 모두 확정하려다 보니 예상보다 많은 시간이 소요되었다.
- 큰 흐름과 모듈의 책임, 입출력 조건을 먼저 정의하고 세부 구현은 개발 과정에서 결정하는 방식이 더 효율적이라는 점을 배웠다.
- 기획 단계에서는 문서화 과정의 필요성이 잘 느껴지지 않았지만, 실제 구현 과정에서 설계 내용을 기준으로 작업하니 설계서 없이 개발할 때보다 구현 속도가 빨라지는 것을 체감했다.
- 설계와 실제 배선 또는 구현이 달라질 경우, 차이를 단순히 남겨두는 것이 아니라 변경 이유와 최종 기준을 문서에 함께 기록해야 협업 과정의 혼란을 줄일 수 있다는 점을 배웠다.

개인 트러블 슈팅 및 후기(조성수)

Trouble Shooting: 타이머 시작 시 의도치 않은 인터럽트 발생 문제

문제 증상

타이머를 시작할 때(`Timer2_Start`), 타이머 카운팅이 정식으로 시작되기도 전에 **시작과 동시에 인터럽트가 즉시 발생하는 현상**이 발생.

원인 분석

MCU 타이머에서 `EGR(Event Generation Register)`의 0번 비트(UG, Update Generation)를 `1`로 세트하면 레지스터 적재가 일어나면서 **`SR(Status Register)`의 0번 비트(UIF, Update Interrupt Flag)가 자동으로 `1`로 세트**.

- **기존 코드의 문제 발생 순서:**
  1. `SR` 비트를 먼저 0으로 지움 (`Macro_Clear_Bit(TIM2->SR, 0)`)
  2. `EGR`을 세트함 → **이 순간 `SR`의 0번 비트가 다시 `1`로 켜짐.**
  3. `DIER`을 설정해 인터럽트를 허용함 → `SR`의 플래그가 `1`인 상태이므로 **즉시 인터럽트 실행**.

> **비유로 이해하기**
> `EGR`을 건드리는 것은 **"새 설정을 적용하면서 공사 먼지를 만드는 작업"**과 같음.
> 먼지(`SR` 플래그)를 먼저 닦아낸 다음 공사(`EGR` 세트)를 시작하면 다시 먼지가 쌓임. 따라서 **공사(`EGR`)를 먼저 마친 후 먼지(`SR`)를 닦아내고, 마지막에 앰프(`DIER`)를 켜야** 소음 없이 깨끗하게 작동.

코드 비교

문제 코드 (Before)

`EGR`을 세트하기 전에 `SR`을 먼저 지우는 바람에, `EGR` 실행 시 발생한 플래그가 그대로 남아 인터럽트가 즉시 터지는 구조.

```
void Timer2_Start(void)
{
    // EGR 세트 전에 SR을 지움 (뒤에서 다시 1로 켜지므로 의미 없음)
    Macro_Clear_Bit(TIM2->SR, 0);

    // EGR 세트 시 Update Event 발생 -> SR의 UIF 비트가 다시 1로 세트됨!
    Macro_Set_Bit(TIM2->EGR, 0);

    NVIC_ClearPendingIRQ(28);

    // SR에 플래그가 남아있어 인터럽트 허용 직후 즉시 인터럽트가 터짐
    Macro_Set_Bit(TIM2->DIER, 0);
    Macro_Set_Bit(TIM2->CR1, 0);
}

void Timer2_Stop(void)
{
    Macro_Clear_Bit(TIM2->CR1, 0);
    Macro_Clear_Bit(TIM2->DIER, 0);
}
```

해결 코드

`EGR`을 세트하고 난 다음, `SR`을 지움 `EGR` 실행 시 발생한 플래그가 지워져 인터럽트가 즉시 터지지 않게 됨.

```
void Timer2_Start(void)
{
    // 1. EGR 세트 (레지스터 적재 & Update Event 발생)
    Macro_Set_Bit(TIM2->EGR, 0);

    // 2. EGR 실행 여파로 켜진 SR의 UIF 플래그를 지움
    Macro_Clear_Bit(TIM2->SR, 0);

    // 3. NVIC의 대기 중인 인터럽트 클리어
    NVIC_ClearPendingIRQ(28);

    // 4. 안전해진 상태에서 인터럽트 허용 및 타이머 구동
    Macro_Set_Bit(TIM2->DIER, 0);
    Macro_Set_Bit(TIM2->CR1, 0);
}

void Timer2_Stop(void)
{
    Macro_Clear_Bit(TIM2->CR1, 0);
    Macro_Clear_Bit(TIM2->DIER, 0);
}
```

개인 트러블 슈팅 및 후기(김형진)

트러블 슈팅

목차

- 트러블 슈팅
  - 1. 모터의 상태값을 정의했지만 제어기준이 모호해지는 문제
  - 2. 인터럽트가 아얘 발생을 안하는 문제
  - 3. UART로 속도 입력시 오작동 하는 문제
  - 4. 버튼을 눌렀을때 타이머2가 작동을 안하는 문제
- 후기 / 느낀점

---

1. 모터의 상태값을 정의했지만 제어기준이 모호해지는 문제

1. 문제상황
   - 처음에 모터의 상태를 정의 할 때 버튼을 누를 때 마다 이전의 상태 동작에 따라 반대방향으로 변화 해야 한다고 정의 했고
   - 버튼을 누를 때마다 모터가 이전 동작의 반대 방향으로 전환되도록 상태 제어 로직을 구상함.
   - 변수 설계 시 다음과 같이 prev를 기준으로 다음 동작을 결정하고 curr을 단순 기록용으로 사용하는 방식으로 구현함.

```
(prev == MOTOR_STOP) -> curr = 이전에 저장된 값
                        prev = 이전에 저장된 값

(prev == MOTOR_CW)   -> curr = MOTOR_CCW
                        prev = MOTOR_CCW

(prev == MOTOR_CCW)  -> curr = MOTOR_CW
                        prev = MOTOR_CW
```

이로 인해 현재 모터의 상태와 변수값이 직관적으로 매핑되지 않아, 모터 속도 제어 및 UART 통신 입력 처리 시 로직이 꼬이고 제어 기준을 잡기 어려워짐.

2. 문제원인
   - 변수명과 역할의 불일치: prev(과거/이전)와 curr(현재)라는 변수명이 주는 시점의 의미와 실제 코드상의 역할이 반대로 매핑되어 있었음. (curr을 단순 기록용, prev를 제어 결정용으로 사용)
   - 제어 기준점의 모호함: "현재 모터가 어떻게 동작하고 있는가?"가 아닌 "이전에 무엇을 했는가?"를 기준으로 다음 동작을 결정하려다 보니, 과거-현재-미래 시점이 혼재되어 코드의 가독성과 제어 직관성이 떨어짐.

3. 해결방법

```
    if(curr_state == MOTOR_STOP){ // 현재 돌아가는 상태 기준

        if(prev_state == MOTOR_STOP){
            curr_state = MOTOR_CW;
            prev_state = MOTOR_CW;
        }
        else if(prev_state == MOTOR_CW){
            curr_state = MOTOR_CW;
            prev_state = MOTOR_STOP;
        }
        else if(prev_state == MOTOR_CCW){
            curr_state = MOTOR_CCW;
            prev_state = MOTOR_STOP;
        }

    }

    else if(curr_state == MOTOR_CW){// 현재 돌아가는 상태 기준

        prev_state = MOTOR_CW;
        curr_state = MOTOR_CCW;
    }
    else if(curr_state == MOTOR_CCW){// 현재 돌아가는 상태 기준

        prev_state = MOTOR_CCW;
        curr_state = MOTOR_CW;
    }
```

제어의 기준을 현재 사용자가 보고 있는 모터의 동작 상태(curr)로 통일"하도록 직관적인 상태 제어 모델로 재정의함.

- 상태 결정 기준 전환:

  이전 상태(prev)가 아닌 현재 동작 상태(curr)를 최우선 기준점으로 설정.

  예: 현재 모터가 시계 방향(curr = MOTOR_CW)으로 돌고 있는 상태에서 버튼이 눌리면, 현재 상태를 기준 삼아 반대 방향(MOTOR_CCW)으로 전환.

---

2. 인터럽트가 아얘 발생을 안하는 문제

인터락으로 구현하고 인터럽트로 바꾸는 과정

1. 문제상황
   - 버튼 입력 또는 타이머/UART 동작 시 인터럽트가 아예 발생하지 않거나(함수가 호출되지 않음), 시스템이 멈추는 현상이 발생함.
   - 디버깅 결과, C 코드상에서 인터럽트 제어 로직과 ISR 함수는 문제없이 작성되어 있었으나 실행 시점에 인터럽트 핸들러로 진입하지 못함.
2. 문제원인
   - 스타트업 파일(.s)과의 함수명 불일치:

     임베디드 MCU(ARM Cortex-M 등)에서는 스타트업 파일(.s)의 벡터 테이블에 인터럽트 발생 시 Jump할 핸들러 함수 이름(예: EXTI0_IRQHandler, USART1_IRQHandler 등)이 약속된 약어로 미리 선언되어 있음.

   - C 코드에서 ISR 함수 이름을 작성할 때, 스타트업 파일에 정의된 이름이 아닌 임의로 지정한 이름(예: my_key_isr())으로 작성함.
3. 해결방법
   - 스타트업 파일의 Vector Table 이름과 C 코드 함수명 동기화

---

3. UART로 속도 입력시 오작동 하는 문제

1. 문제상황
   - 모터가 이미 특정 방향(예: MOTOR_CW)으로 정상 회전 중일 때, UART 통신으로 속도 변경 명령을 전송함.
   - 모터가 회전 상태를 유지하면서 속도만 변경되어야 하지만, 모터 제어 메인 루프(Motor_Main)가 속도 명령을 방향 전환 명령으로 인지하여 동작이 정지/재시작되는 오작동 발생.
2. 문제원인
   - 상태 변수 미 갱신
3. 해결방법
   - running_state 상태 기록 변수 추가:
   - 모터가 실제로 돌고 있는지, 정지했는지를 기억하는 static MotorState running_state 변수를 추가하여 상태 변경 여부 판단 기준으로 사용.
   - 이미 동일한 방향으로 회전 중인 상태(running_state == curr_state)일 경우, Motor_Main() 상단에서 빠르게 빠져나가도록 처리하여 단순 PWM 속도값만 즉시 반영되도록 구현.

UART로 속도 입력시 현재방향 그대로 속도가 변화해야하는데 속도 변경하고 함수에 들어와서 속도 입력시 오작동 하는 문제점 모터가 돌고있을때 현재 돌고있는 상태 그대로라서 방향은 넘어가야하는데 방향바꾸러 들어온다는 점 상태의 업데이트 안해서 Uart입력시 오작동

---

4. 버튼을 눌렀을때 타이머2가 작동을 안하는 문제

1. 문제상황
   - 버튼 입력 인터럽트 발생 시, 3초를 측정하기 위해 Timer2_Start()를 호출하여 타이머2를 시작함.
   - 3초를 정상적으로 카운트한 뒤 인터럽트가 발생해야 하지만, 타이머를 시작하자마자 3초를 세지 않고 즉시 타임아웃 인터럽트(ISR)로 진입하는 현상 발생.
2. 문제원인
   - EGR (Event Generation Register)의 부작용:

     타이머 시작 시 CNT 및 PSC를 (다운 카운트) 큰값부터 정확히 세기 위해 TIM2->EGR의 UG 비트를 1로 세팅함.

     그러나 STM32 구조상 EGR 비트를 세팅하면 갱신 이벤트가 소프트웨어적으로 강제 발생하면서 SR의 UIF 가 자동으로 1로 세팅됨.

   - 인터럽트 트리거 꼬임:

     만약 DIER 비트가 켜져 있는 상태에서 EGR을 건드리거나, SR 플래그를 지우는 순서가 꼬이면, 타이머 카운팅이 시작되기도 전에 EGR이 만든 거짓 플래그 때문에 즉시 ISR로 튕겨 들어가는 문제가 발생함.

```
void TIM2_Interrupt_Enable(int en){
    if(en){
        // TIM4 Pending Clear
        Macro_Clear_Bit(TIM2->SR, 0);
```

```
        // NVIC Pending Clear
        NVIC_ClearPendingIRQ(28);
        // TIM4 Interrupt Enable
        Macro_Set_Bit(TIM2->DIER, 0);
        // NVIC Interrupt Enable
        NVIC_EnableIRQ(28);
        Macro_Set_Bit(TIM2->CR1, 0);

    }
    else{
        NVIC_DisableIRQ(28);
        Macro_Clear_Bit(TIM2->CR1, 0);
        Macro_Clear_Bit(TIM2->DIER, 0);
    }
}

void Timer2_Start()
{
    Macro_Set_Bit(TIM2->EGR,0);
    Macro_Clear_Bit(TIM2->SR, 0);
    Macro_Set_Bit(TIM2->CR1, 0);
}
```

3. 해결 방법
   - EGR로 카운터를 초기화한 후, SR 플래그를 깔끔히 지우고 나서 DIER 및 CR1을 켜도록 레지스터 설정 순서 재정렬

```
void Timer2_Start(void)
{
    // 1. DIER을 먼저 꺼서 EGR에 의한 가짜 인터럽트가 NVIC로 전달되는 것을 방지
    Macro_Clear_Bit(TIM2->DIER, 0);

    // 2. EGR 비트를 세팅하여 CNT 및 프리스케일러 0으로 강제 초기화 (이때 SR의 UIF가 1이 됨)
    Macro_Set_Bit(TIM2->EGR, 0);

    // 3. EGR이 발생시킨 가짜 갱신 플래그(SR)를 반드시 지움!
    Macro_Clear_Bit(TIM2->SR, 0);

    // 4. 깨끗해진 상태에서 타이머 카운터(CR1)와 인터럽트(DIER)를 비로소 시작!
    Macro_Set_Bit(TIM2->CR1, 0);
    Macro_Set_Bit(TIM2->DIER, 0);
}
```

후기 / 느낀점

상위 설계서를 팀원과 같이 고민하면서 설계를 했는데 지금까지 머리속에서 주먹구구 식으로만 코드를 짜왔었기 때문에 어디서부터 어디까지 설계를 해야하고 정해야하는지 기준을 잘 몰랐었고 하드웨어의 동작을 코드로 구현하는게 지금까지 내가 코드를 짰던 관점과는 많이 달라서 어려웠다

예를들어 레지스터를 비트연산으로 세팅해야되고, 인터럽트를 미리키면 안되고, 주파수를 설정하는 것, 보드의 출력을 10 으로 설정했으면 핀을 어떻게 정해야하는지 핀의 역할도 여러개다보니 헷갈리는 부분이 굉장히 많았다

그래서 팀원들과 계속 물어보고 설계의 방향성을 같이 이야기하다보니 구현까지는 완료가 되었다

시간관계상 아직 구현하지 못한 방향 전환할때 모터를 정지하고 해야하는 부분이나
모터로직 안에서 타이머가 쓰이는 의존성 관리등을 수정하지만
추후에 더 개선해 나가야 할 것 같고
다음 프로젝트에서는 어떤 방향으로 설계를 해야 하는지 설계의 깊이같은 것에 감이 잡힌것 같은 생각이 들었다
